

Name : R. Indhu

CO2 - ASSESSMENT - 1

Analytical Problems

Register No: 198524332

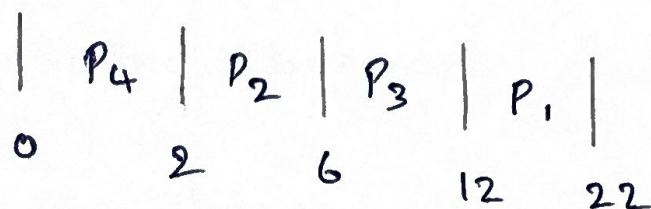
Course: Operating Systems

Course Code: 08CSA0404

1. CPU Scheduling - SJF (Non-Preemptive):

Process	Burst Time (ms)	Completion Time (CT)
P1	10	22
P2	4	12
P3	6	6
P4	2	2

Gantt Chart:



Turn Around Time (TAT):

$$TAT = CT - AT \quad (\text{or}) \quad WT + BT$$

WT [Waiting Time]:

$$WT = TAT - BT$$

Process	BT	CT	TAT	WT
P4	2	2	2	0
P3	6	6	12	6
P2	4	12	6	2
P1	10	22	22	12

$$\text{Average TAT} = \frac{2+6+12+22}{4} = \frac{42}{4}$$

$$= 10.5$$

$$\text{Average WT} = \frac{0+2+6+12}{4} = \frac{20}{4}$$

$$= 5$$

2. Round Robin Scheduling:

Process Burst Time (BT)

P1 20

P2 5

P3 13

P4 8

Gantt Chart:

| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P1 |

0 4 8 12 16 20 21 25 29 33 37 42 46

Process	BT	CT	TAT	WT
P1	20	46	46	26
P2	5	21	21	16
P3	13	37	37	24
P4	8	29	29	21

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$\text{Average TAT} = \frac{46 + 21 + 37 + 29}{4}$$

$$\text{Average WT} = \frac{26 + 16 + 24 + 21}{4}$$

Effect of Increasing Time Quantum to 10ms

- * Content switching decreases.
- * CPU utilization improves.
- * Response time for small processes increases.
- * Scheduling becomes similar to FCFS.

3. Banker's Algorithm:

Need = Maximum - Allocation.

Process Need (A B C)

P1 7 4 3

P2 1 2 2

P3 6 0 0

P4 2 1 1

P5 5 3 1

Step 2: Check safe state

Processes that can execute

The deadlock can be resolved

$P_2 \longrightarrow P_4 \longrightarrow P_5 \longrightarrow P_1 \longrightarrow P_3$

Step 3: Safe Sequence

$P_2 \longrightarrow P_4 \longrightarrow P_5 \longrightarrow P_1 \longrightarrow P_3$

4. Resource Allocation Graph (Deadlock) Detection)

Given,

- P_1 holds R_1 & requests R_2 .
- P_2 holds R_2 & requests R_3 .
- P_3 holds R_3 & requests R_1 .

Step 1: Draw Resource Allocation Graph

$P_1 \longrightarrow R_2$	$P_2 \longrightarrow R_3$	$P_3 \longrightarrow R_1$
$R_1 \longrightarrow P_1$	$R_2 \longrightarrow P_2$	$R_3 \longrightarrow P_3$

Step 2: Check for Cycle:

$P_1 \longrightarrow R_2 \longrightarrow P_2 \longrightarrow R_3 \longrightarrow P_3 \longrightarrow R_1 \longrightarrow P_1$

* Since a cycle exists, the system is in deadlock.

Step 3: Deadlock Recovery

Possible recovery methods:

- Terminate one of the deadlocked processes
- Preempt a resource from one process.
- Restart the affected process.

$\Rightarrow$ The Resource Allocation Graph contains a cycle, indicating a deadlock.

Cycle: $P_1 \longrightarrow P_2 \longrightarrow P_3 \longrightarrow P_1$

The deadlock can be resolved by terminating a process or preempting resources.

5. CPU Utilization in Multiprogramming

Given

- Probability that a process is waiting for I/O (p) = 0.8
- Degree of Multiprogramming (n) = 4

$$\text{CPU Utilization} = 1 - p^n$$

Step 1: Calculate CPU Utilization for $n = 4$

$$U = 1 - (0.8)^4$$

$$U = 1 - 0.4096 = 0.5904$$

$$\text{CPU Utilization} = 59.04\%$$

Step 2: Calculate CPU Utilization for $n = 6$

$$U = 1 - (0.8)^6$$

$$U = 1 - 0.2621 = 0.7379$$

$$\text{CPU Utilization} = 73.79\%$$

Step 3: Interpretation

- Increasing the degree of Multiprogramming improves CPU utilization.
- More processes keep the CPU busy while others wait for I/O.
- Hence, $n = 6$ gives better performance than $n = 4$.

6. Producer - Consumer Problem Using Semaphores

Given

- Buffer Size = 6
- Empty = 6
- Full = 0
- Mutex = 1

Step 1: After 3 Producer Operations

Each producer fills one buffer slot.

- Empty = 3
- Full = 3
- Mutex = 1

Step 2: After 2 Consumer Operations.

Each consumer removes one item.

- Empty = 5
- Full = 1
- Mutex = 1

Step 4: Role of Mutex

- Mutex allows only one process to access the buffer at a time.
- It prevents race conditions.
- It ensures data consistency.

Step 4: Without Mutex:

- Multiple processes may access the buffer simultaneously.
- Data inconsistency may occur.
- Race conditions can lead to incorrect results.

7. Critical Section Problem

Given

- Initial value of $x = 100$
- Two processes increment x 1000 times each.

Step 1: Race Condition:

If both processes update x simultaneously without synchronization, some increments may be lost.

Step 2: Correct final Value

Each process increments x 1000 times:

Total increments = 2000

Final Value:

$$100 + 2000 = 2100$$

Correct Value = 2100

Step 3: Hardware Solution

Test - and - Set Instruction

- Ensures only one process enters the critical section at a time.
- Prevents race conditions.

Step 4: Software Solution

- Semaphore
- Peterson's Algorithm

These methods ensure mutual exclusion and proper synchronization.

8. Thread Scheduling Models

Given,

A system has 8 CPU cores. Compare Many-to-One and One-to-One thread models & identify the better model.

Step 1: Many to One Model

- Many user threads are mapped to one kernel thread.
- Only one thread executes at a time.
- Parallel execution is not possible
- CPU utilization is low.

Step 2: One to One Model

- Each user thread is mapped to one kernel thread.
- Multiple threads execute simultaneously.
- Supports parallel processing on multi-core systems.
- CPU utilization is high.

Step 3: Comparison:

Many-to-One

One kernel thread

No parallel execution

Low CPU utilization

Slower performance

One to One

One kernel thread per user thread

Supports parallel execution.

High CPU utilization

Better performance.

Step 1: Deadlock Prevention

- Pre...

9. Monitor Synchronization

Given,

A monitor is used for synchronization between multiple processes.

Step 1: Mutual Exclusion

- A monitor allows only one process to enter the critical section at a time.
- Other processes wait until the monitor becomes free.
- This prevents race conditions.

Step 2: wait() and signal() Operations.

- wait(): Suspends a process until a condition becomes true.
- signal(): Wakes up one waiting process.

Step 3: without signal()

- Waiting processes remain blocked
- The required resource is never allocated.
- This may lead to indefinite waiting.

10. Deadlock Prevention vs Deadlock Avoidance.

Given,

Compare Deadlock Prevention and Deadlock Avoidance with examples.

Step 1: Deadlock Prevention

- Prevents deadlock by eliminating one of the necessary conditions.
- Resources are allocated using fixed rules.
- Example: Preventing the Hold and Wait Condition.

Step 2: Deadlock Avoidance

- Allocates resources only if the system remains in a safe state.
- Uses Banker's Algorithm.
- Avoids deadlock before it occurs.

Step 3: Comparison.

Deadlock Prevention

Eliminates deadlock conditions

Simple to implement

Lower resource utilization

Example: No Hold and Wait

Deadlock Avoidance.

Checks for safe state before allocation.

More complex

Better resource utilization

Example: Banker's Algorithm.